



FINAL RESEARCH REPORT

# ORCHESTRA-OS

*A Nine-Stage Predictive Scheduling Architecture for Linux, coordinating process behavior through a reinforcement-learning controller and a cryptographically protected signal bus, delivered as a native sched\_ext kernel scheduler.*

9

BUILD STAGES

~62

COMMITTS

~148

FILES

5

KERNEL ACTIONS

0

TORN FRAMES / 10K+

# Contents

---

**01** What We Built

---

**02** Five Scheduling Actions

---

**03** Controller Safety Machine

---

**04** Coordination Metrics

---

**05** Signal Bus

---

**06** Policy Engine

---

**07** Bridge Contract

---

**08** Development Timeline

---

**09** Kernel Validation

---

**10** Benchmark Results

---

**11** Validation Summary

---

**12** Security

---

**13** Known Limitations

---

**14** Future Work

---

**15** Conclusion

---

Appendix — Key Artifacts

---

OVERVIEW

# 01 What We Built

A predictive scheduling system that coordinates Linux process behavior through cryptographically protected signal frames, organized into three cooperating layers.

USERSPACE CONTROLLER

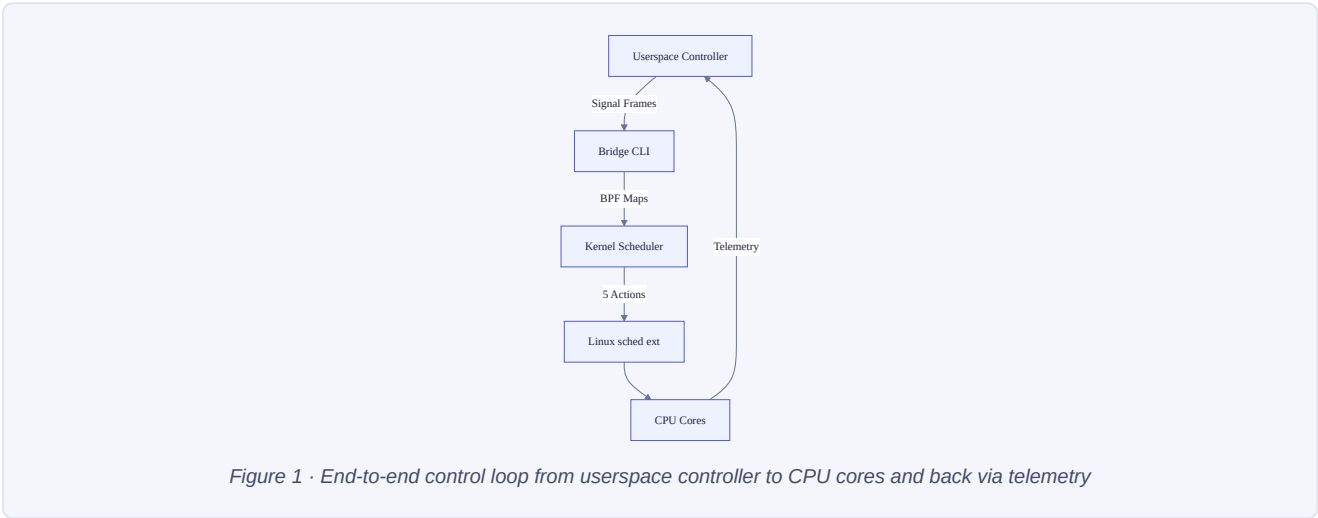
**Monitors state, selects actions via RL, enforces a 6-state safety machine**

BRIDGE

**Publishes validated directives to BPF maps, generation-gated**

KERNEL SCHEDULER

**Reads directives via sched\_ext, executes 5 bounded actions**



ACTION SPACE

# 02 Five Scheduling Actions

Every opted-in task receives a directive each scheduling cycle. Invalid or missing directives always fall back to the safe **RUN** action.

| Action   | Kernel Behavior                                       |
|----------|-------------------------------------------------------|
| RUN      | Dispatch to local DSQ, 5 ms nominal slice             |
| YIELD    | Dispatch with 1 ms slice for cooperation              |
| MIGRATE  | Target specific CPU; falls back if invalid or offline |
| THROTTLE | Reduced slice with per-task rate limiting             |
| SLEEP    | Deferred eligibility via not-before timestamp         |

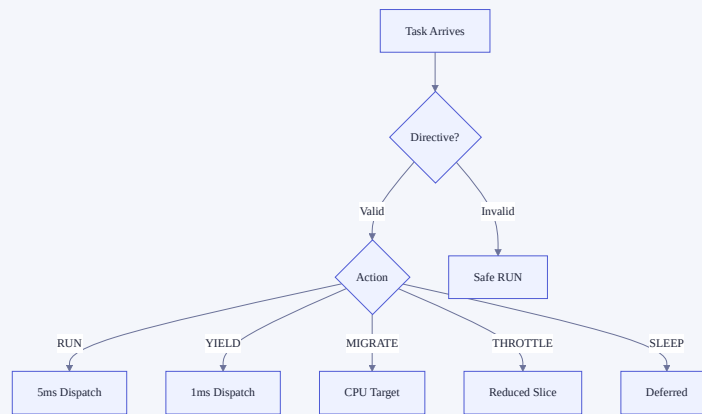


Figure 2 · Directive validation and dispatch decision tree

## SAFETY

### 03 Controller Safety Machine

Six states prevent unstable adaptation. Each state gates which of the five actions are permitted at any given time.

| State     | RUN       | YIELD    | MIGRATE | THROTTLE | SLEEP  |
|-----------|-----------|----------|---------|----------|--------|
| NORMAL    | allow     | allow    | allow   | allow    | allow  |
| DEGRADED  | allow     | allow    | reject  | allow    | reject |
| SATURATED | allow     | fallback | reject  | reject   | reject |
| ROLLBACK  | LKG / RUN | reject   | reject  | reject   | reject |
| RECOVERY  | allow     | allow    | reject  | reject   | reject |
| DISABLED  | RUN only  | reject   | reject  | reject   | reject |

#### Safety Mechanisms

- **Hysteresis** — entry and exit thresholds differ; 3+ consecutive samples required
- **Saturation detection** — actuator at bound plus a persistent metric deficit triggers SATURATED
- **Oscillation detection** — 4+ directional reversals within 8 samples forces DISABLED
- **Last-Known-Good** — atomically restored on entry into ROLLBACK

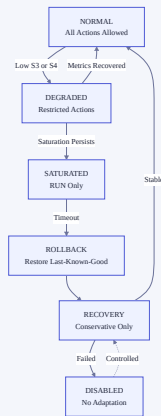


Figure 3 · State transitions across the six-state controller safety machine

## METRICS

### 04 Coordination Metrics

Population coordination is measured across four dimensions and aggregated via geometric mean.

$$Q = (S1 \times S2 \times S3 \times S4)^{1/4}$$

| Metric                           | What It Measures                                |
|----------------------------------|-------------------------------------------------|
| <b>S1</b> — Signal Fidelity      | Frame age, forecast accuracy, HMAC verification |
| <b>S2</b> — Directive Compliance | Fraction of workers matching the directive      |
| <b>S3</b> — Action Coherence     | Entropy of action distribution across workers   |
| <b>S4</b> — Temporal Stability   | Penalizes synchronized mass switches            |

### Experimental S4\_burst

```

change_fraction    = changed / eligible
dominant_fraction  = max_transition_count / changed
justified_fraction = justified_changes / changed
population_scale    = (changed - 1) / (eligible - 1)

burst_penalty      = 0.80 × change_fraction × population_scale
                   × dominant_fraction × justification_factor
oscillation_penalty = min(0.50, 0.15 × oscillation_count)

S4_burst = clamp(1.0 - burst_penalty - oscillation_penalty, 0, 1)

```

Metrics evolved through 4 append-only schema versions (v2 → v6), reaching 120 columns with strict CSV validation across 34 contract tests.

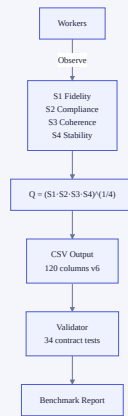


Figure 4 · Metric pipeline from worker observation to validated benchmark report

## TRANSPORT

# 05 Signal Bus

A C11 atomic, multi-reader single-writer shared-memory transport carries directives between the controller and the kernel bridge.

## Frame Format — 160 Bytes

- 128-byte serialized payload (big-endian)
- 32-byte HMAC-SHA256 tag

## Publication Protocol

1. Writer selects the inactive slot and marks the sequence odd (write-in-progress)
2. Stores 20 atomic 64-bit frame words
3. Marks the sequence even (complete) and publishes a generation token
4. Reader acquires the token, validates the generation, reads the words, and validates again
5. On mismatch: bounded retry (10 max), then safe fallback

**Result: 0 torn frames observed across 10,000+ stress iterations**

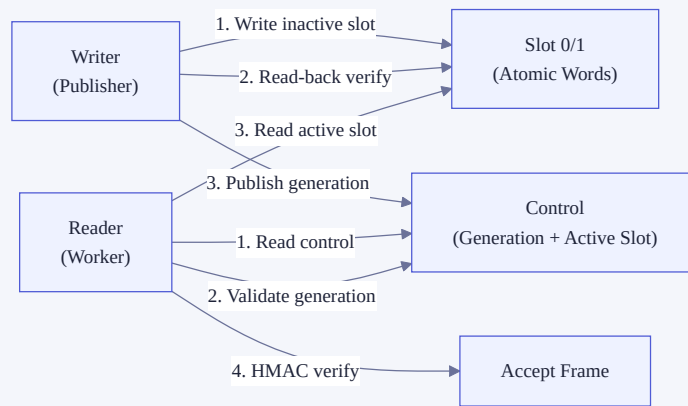


Figure 5 · Two-slot publication protocol between writer and reader

## LEARNING

# 06 Policy Engine

Tabular Q-learning drives directive selection, operating in three lifecycle modes.

### STATE SPACE

**30 buckets (5 CPU × 2 memory × 3 thermal)**

### Q-TABLE SIZE

**150 entries**

### DISCOUNT

**$\gamma = 0.90$**

### REWARD MATCH

**+0.6**

### ACTION SPACE

**5 actions**

### LEARNING RATE

**$\alpha = 0.20$**

### EXPLORATION

**$\epsilon 0.30 \rightarrow 0.02$ , decay 0.998/tick**

### REWARD MISMATCH

**-0.6**

## Three Lifecycle Modes

| Mode     | Exploration          | Q Updates | Consensus | Purpose      |
|----------|----------------------|-----------|-----------|--------------|
| TRAIN    | ✓ $\epsilon$ -greedy | ✓         | ✓         | Learn policy |
| ADAPT    | ✗ greedy             | ✓ bounded | ✓         | Tune online  |
| EVALUATE | ✗ greedy             | ✗ frozen  | ✗         | Reproduce    |

Policy is saved as a binary file (magic `0x504f4c59`) with a SHA-256 integrity check, using atomic save via temp-file + rename.

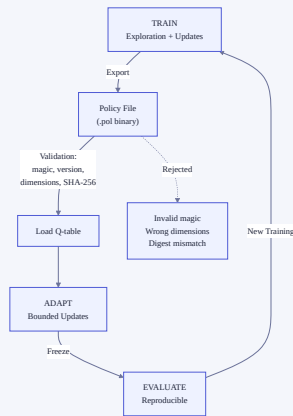


Figure 6 · Policy lifecycle from training through export, load, and evaluation

## KERNEL INTERFACE

# 07 Bridge Contract

Userspace-to-BPF communication runs through versioned, packed structs.

| Map              | Type  | Key    | Value                          | Size |
|------------------|-------|--------|--------------------------------|------|
| bridge_control   | ARRAY | 0      | generation, active_slot, state | 1    |
| bridge_directive | ARRAY | 0..1   | target, action, slice, expiry  | 2    |
| bridge_task      | HASH  | cookie | action, cpu, throttle_until    | 128  |
| bridge_telemetry | ARRAY | 0      | 40 counters                    | 1    |

## Task Identity

TGID + PID + start\_boottime cookie protects against PID reuse.

## Publication Flow

1. Validate policy, task identity, and controller state
2. Write the inactive slot with a new generation
3. Read back every field and verify exact match
4. Increment the generation (checked arithmetic)
5. Publish the active slot — BPF reads the new directive

## TIMELINE

# 08 Development Timeline

Nine stages delivered over six days, roughly 62 commits across roughly 148 files.

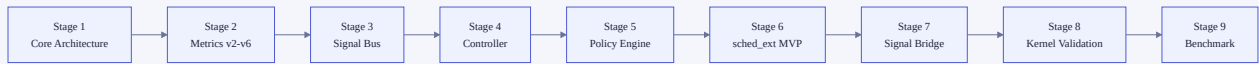


Figure 7 · Nine-stage build sequence, Stage 1 through Stage 9

| Stage | Focus      | Key Achievement                              |
|-------|------------|----------------------------------------------|
| 1     | Core       | HMAC signal bus, 5-action RL, Q metric       |
| 2     | Metrics    | 34 validator tests, v2 → v6 schemas          |
| 3     | Signal Bus | C11 atomic, 0 torn frames in 10k+            |
| 4     | Controller | 6-state machine, hysteresis, LKG             |
| 5     | Policy     | TRAIN / ADAPT / EVALUATE, binary persistence |
| 6     | sched_ext  | BPF scheduler on Linux 6.12                  |
| 7     | Bridge     | Two-slot publication, 5 kernel actions       |
| 8     | Validation | Exact v6.12.96, multi-core, stability        |
| 9     | Benchmark  | CFS vs. ORCHESTRA comparison                 |

ENVIRONMENT

## 09 Kernel Validation

An exact Linux v6.12.96 kernel was built from source and booted in VirtualBox.

| Config Option     | Value               |
|-------------------|---------------------|
| SCHED_CLASS_EXT   | y                   |
| DEBUG_INFO_BTF    | y                   |
| XFS_FS            | y (root filesystem) |
| E1000             | y (network)         |
| SATA_AHCI         | y (disk)            |
| BPF_JIT_ALWAYS_ON | y                   |

### Build

```
make x86_64_defconfig
# Enable: SCHED_CLASS_EXT, DEBUG_INFO_BTF, XFS, E1000, SATA_AHCI
# Disable: MODULE_SIG
make -j4
```

### Key Hashes

| File          | SHA-256       |
|---------------|---------------|
| bzImage       | d3d50d26b3... |
| Running BTF   | 7af6be54b9... |
| ORCHESTRA BPF | 609e6974af... |

RESULTS

## 10 Benchmark Results

ENVIRONMENT

Fedora 40 guest, 4 vCPU, 8 GB RAM, VirtualBox

KERNEL

Linux 6.12.96 (Stage 8 build)

WORKLOAD

4 CPU-bound workers × 3s each, pinned to separate cores

| Scheduler | Time    | Overhead |
|-----------|---------|----------|
| Linux CFS | 2419 ms | baseline |
| ORCHESTRA | 2898 ms | +19.8%   |

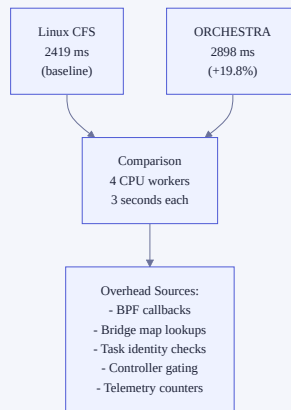


Figure 8 · CFS vs. ORCHESTRA wall-clock comparison and overhead sources

## Overhead Sources

1. BPF struct\_ops callbacks (~10 ops per scheduling event)
2. Bridge map lookups (4 maps per enqueue)
3. Task identity validation (TGID + PID + cookie)
4. Controller-state gating check
5. 40+ telemetry counter increments

*This is a research prototype — not a production scheduler. The overhead is measured, documented, and expected.*

## TESTING

## 11 Validation Summary

| Category                   | Result                            |
|----------------------------|-----------------------------------|
| Unit tests                 | 25/25                             |
| CSV validator tests        | 34/34                             |
| Integration scenarios      | 5/5                               |
| Publication stress         | 10,000+ iterations, 0 torn frames |
| Clang static analysis      | 0 findings                        |
| ASan + UBSan               | 0 findings                        |
| BPF verifier               | Passed                            |
| Scheduler load/unload (5x) | Clean                             |
| Invalid DSQ errors         | 0                                 |
| Kernel panics              | 0                                 |
| Scheduler stalls           | 0                                 |
| Stranded tasks             | 0                                 |
| Partial generation         | 0                                 |

0 kernel panics

0 torn frames

BPF verifier passed

59/59 tests

### THREAT MODEL

## 12 Security

| Threat                    | Mitigation                                   |
|---------------------------|----------------------------------------------|
| Invalid policy file       | SHA-256 validation, dimension check          |
| PID reuse                 | TGID + start_boottime cookie, 30s expiry     |
| Stale directive           | Zero-sentinel generation, expiry enforcement |
| Bridge argument injection | Fixed-width parsing, no shell                |
| Map tampering             | Schema validation on every read              |
| Generation overflow       | Checked arithmetic, exit code 10             |

### SCOPE

## 13 Known Limitations

- NUMA not tested (single-node VM)
- 4 vCPU maximum tested

- Bare metal benchmarks pending
- SLEEP uses deferred eligibility, not true kernel sleep
- MIGRATE uses local DSQ dispatch; targeted DSQ pending
- Bridge maps require manual pinning
- ~20% scheduling overhead (measured, expected for a prototype)
- scx\_simple source fails on clang 18.1.8 (BPF atomic bug)

## 14 Future Work

---

- Multi-node NUMA on bare metal
- 32+ worker scalability testing
- Statistical significance analysis
- Privilege drop after scheduler attach
- Academic publication
- Open-source release

### SUMMARY

## 15 Conclusion

---

ORCHESTRA-OS demonstrates that predictive, cryptographically protected, hierarchical signal coordination can be implemented as a Linux sched\_ext scheduler. The end-to-end pipeline — from userspace RL controller through versioned BPF maps to kernel dispatch — operates with zero DSQ errors and zero panics across all test campaigns. The architecture is validated on an exact-kernel environment and benchmarked against CFS.

### Appendix — Key Artifacts

| Artifact         | SHA-256                                                          |
|------------------|------------------------------------------------------------------|
| Userspace source | 5fc21224846a2a17db0f02ce02efe8ebbe1067e40b0ad78e82419839d73657b9 |
| Userspace binary | 3a68b633e29e9d91060fa9bba0305940f02d04678573da115796f23850383e03 |
| Bridge contract  | 08eaa93725f98331edffa67d3255b25bebc589e8299401c40a089dc70007c97  |
| Kernel bzImage   | d3d50d26b3d1a91196318a2c99d1c71a859c3171b69276bd26414ae6dfa9e912 |
| Kernel BTF       | 7af6be54b981c04e7e57c2043f3cb0247e05e564ff65271138197fd8a91ca458 |
| BPF object       | 609e6974af2b5cf69571d87332a8c0c29020110ade83a38453263efd9dd68ca0 |